



BURSA TEKNİK ÜNİVERSİTESİ

MÜHENDİSLİK VE DOĞA BİLİMLERİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

DERSİN ADI : BİLGİSAYAR AĞLARI

PROJE KONUSU: NetProbe: UDP Tabanlı Güvenilir Dosya Aktarımı,
Trafik İzleme ve Ağ Performans Analiz Platformu

ÖĞRENCİ NUMARASI: 22360859032

ÖĞRENCİ ADI SOYADI: Muhammed Fatih Göral

TESLİM TÜRÜ: Rapor ve Kaynak Kodları

İçindekiler

1. Giriş
 - 1.1. Projenin Amacı
2. Problem Tanımı
3. Sistem Mimarisi
 - 3.1. Modül Yapısı
 - 3.2. Veri Akışı
4. Protokol Tasarımı
 - 4.1. Paket Tipleri
 - 4.2. DATA Paket Formatı
 - 4.3. ACK Paket Formatı
 - 4.4. Güvenilirlik Mekanizmaları
 - 4.5. Hata Durumları
5. Gerçekleme Detayları
 - 5.1. İstemci Akışı
 - 5.2. Sunucu Akışı
 - 5.3. Loglama
 - 5.4. Loss Simülatörü
6. Deney Ortamı
7. Performans Metrikleri
8. Deneysel Senaryolar ve Sonuçlar
 - 8.1. Senaryo 1 – Paket Boyutu Etkisi
 - 8.2. Senaryo 2 – Timeout Değeri Etkisi
 - 8.3. Senaryo 3 – Simüle Paket Kaybı Oranı Etkisi
 - 8.4. Senaryo 4 – Dosya Boyutu Etkisi
9. Sonuçların Tartışılması
 - 9.1. Parametre Etkileşimleri
 - 9.2. Föy Gereksinimlerine Uyum
 - 9.3. Sınırlamalar
10. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları
11. Sonuç ve Gelecek Çalışmalar
12. Kaynaklar

1. Giriş

Modern bilgisayar ağıları, taşıma katmanında temel olarak iki farklı protokol sunar: TCP (Transmission Control Protocol) ve UDP (User Datagram Protocol). TCP, bağlantı yönelimli yapısı sayesinde güvenilir veri aktarımını yerleşik olarak sağlarken; UDP, bağlantısız ve minimum başlık taşıyan yapısı nedeniyle düşük gecikme gerektiren uygulamalarda tercih edilir. Ancak UDP'nin doğasında güvenilirlik, sıralama, akış kontrolü ve tıkanıklık kontrolü gibi mekanizmalar bulunmaz; bu işlevlerin uygulama katmanında geliştirilmesi gerekir.

Bu çalışmada geliştirilen NetProbe sistemi, UDP üzerinde stop-and-wait yaklaşımı temelli güvenilir dosya aktarımı sağlayan; eş zamanlı olarak trafik kayıtlarını üreten ve sistem performansını farklı koşullar altında ölçen tam kapsamlı bir platformdur. Platform; istemci, sunucu, protokol motoru, kayıt (logger) modülü, metrik hesaplayıcı, paket kaybı simülatörü ve dört bağımsız deneysel senaryo motorundan oluşmaktadır.

Projenin temel motivasyonu, yalnızca çalışan bir dosya aktarım aracı üretmek değil; geliştirilen güvenilirlik mekanizmasının ağ koşullarındaki davranışını sayısal olarak belgelemek ve protokol parametrelerinin (paket boyutu, timeout süresi, kayıp oranı, dosya boyutu) performans metrikleri üzerindeki etkisini deneysel olarak incelemektir.

1.1. Projenin Amacı

- UDP soket programlama kullanarak istemci-sunucu mimarisi geliştirmek.
- UDP üzerinde sequence number, ACK, timeout ve retransmission tabanlı güvenilir aktarım mekanizması tasarlamak.
- Aktarım sırasında oluşan tüm olayları (gönderim, ACK, timeout, retry, duplicate, hata) zaman damgalı olarak kaydetmek.
- Throughput, goodput, packet loss rate, retransmission rate, completion time ve RTT gibi metrikleri otomatik hesaplamak.
- Sistemi en az dört farklı deneysel senaryo altında test edip sonuçları teknik olarak yorumlamak.
- Dosya bütünlüğünü SHA-256 hash karşılaştırması ile uçtan uca doğrulamak.

2. Problem Tanımı

UDP, başlık alanı yalnızca 8 bayt uzunluğunda olan ve teslimat garantisi içermeyen datagram tabanlı bir protokoldür. UDP üzerinden bir dosyanın eksiksiz ve doğru sırada iletilebilmesi için uygulama katmanında en az şu altı problemin çözülmesi gerekir:

- **Paket kaybı tespiti:** Gönderilen bir paketin alıcıya ulaşp ulaşmadığının anlaşılması.
- **Sıralama:** Paketlerin gönderim sırası ile alım sırasının farklı olabilmesi; doğru sıraya konulması.
- **Yineleme (duplicate) yönetimi:** Aynı paketin birden fazla kez teslim alınabilmesi; alıcıda mükerrer yazıma yol açmaması.
- **Zaman aşımı yönetimi:** Beklenen ACK belirli bir süre içinde gelmezse paketin yeniden gönderilmesi.
- **Yeniden iletim sınırı:** Sonsuz döngünün önlenmesi için maksimum deneme sayısının belirlenmesi.
- **Bütünlük doğrulaması:** Aktarım sonunda dosyanın bit düzeyinde değişmediğinin kanıtlanması.

NetProbe; bu altı problemi paket başlık alanları, durum makineleri ve hash karşılaştırması kullanarak çözmekte ve aynı koşullar altında çalışan ölçüm altyapısı ile sonuçların tekrar üretilebilirliğini sağlamaktadır.

3. Sistem Mimarisi

NetProbe modüller bir mimari ile tasarlanmıştır. Sistem; bir istemci (gönderici), bir sunucu (alıcı), paylaşılan bir protokol katmanı, ortak metrik ve log altyapısı ile dört bağımsız deney koşturucusundan oluşmaktadır. Tüm modüller src/ dizini altında yer alır.

3.1. Modül Yapısı

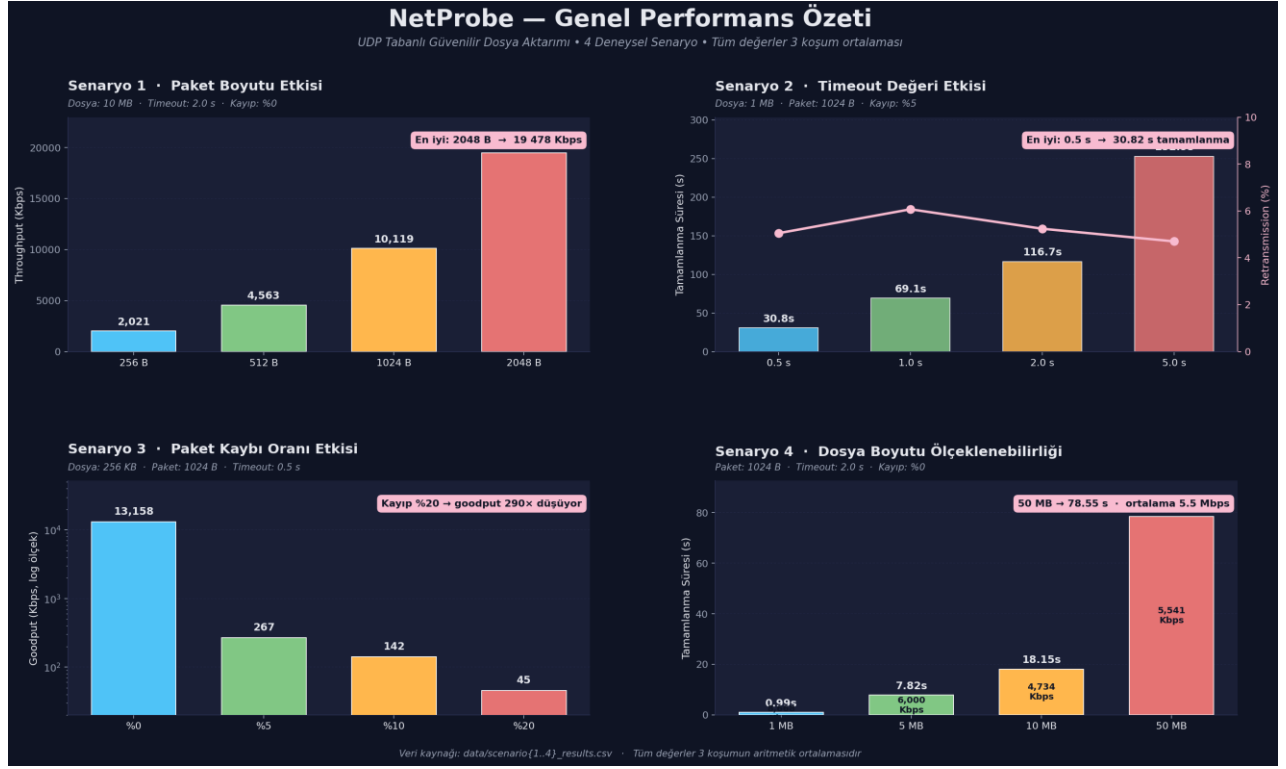
Modül	Görev
src/protocol.py	Paket tipleri (DATA, ACK, START, END, NACK), serileştirme/deserileştirme, SHA-256 checksum.
src/client.py	Dosyayı paketlere bölme, gönderim, ACK bekleme, timeout ve retry yönetimi.
src/server.py	UDP soket üzerinden paket alımı, duplicate kontrolü, dosya yazımı, hash doğrulama.
src/logger.py	Olay tabanlı CSV log üretimi (timestamp, event_type, seq_num, retries, status).
src/metrics.py	Throughput, goodput, RTT, retransmission rate, completion time hesabı.
src/loss_simulator.py	Sunucu tarafında thread-güvenli yapay ACK kaybı simülatörü.
experiments/scenario1..4	Paket boyutu, timeout, kayıp oranı ve dosya boyutu deneyleri.
analysis/	CSV verilerini görselleştirme ve karşılaştırma modülleri.

Tablo 1

3.2. Veri Akışı

İstemci, kullanıcıdan aldığı dosyayı önce tek seferlik bir SHA-256 hash hesabıyla özetler. Ardından dosyayı 1000 baytlık veri payload'larına böler ve her parçayı bir sequence number ile etiketler. Aktarımdan önce START paketi (toplam paket sayısı + dosya hash) gönderilir. Her DATA paketinin ACK'ı alınana kadar istemci o paketi bekler; ACK gelmezse 5 denemeye kadar yeniden gönderim yapılır. Tüm parçalar gönderildikten

sonra END paketi ile sunucuya 5eserve5rin sonu bildirilir. Sunucu, alınan parçaları sırayla dosyaya yazar, son aşamada alınan dosyanın hash'ini hesaplayıp START paketindeki hash ile karşılaştırarak bütünlüğü doğrular.



Şekil 1. NetProbe — tüm senaryoların özet performans karşılaştırması.

4. Protokol Tasarımı

NetProbe, uygulama katmanında çalışan kendi paket formatını tanımlar. Tüm sayısal alanlar ağ bayt sırası (big-endian) ile serileştirilir. Paket tipleri 1 baytlık alan ile ayırt edilir.

4.1. Paket Tipleri

Kod	Tip	Açıklama
0x01	DATA	Veri paketi (payload taşıyan paket)
0x02	ACK	Başarılı alım onayı
0x03	START	Transfer başlangıcı + toplam paket sayısı + dosya hash
0x04	END	Transferin tamamlandığını bildirir
0x05	NACK	Olumsuz onay (6eserve edilmiş)

Tablo 2

4.2. DATA Paket Formatı

DATA paketi, 19 bayt sabit başlık ve en fazla 1000 bayt veri yükü olmak üzere yapılandırılmıştır. Alanların sırası ve boyutları şu şekildedir:

- Paket Tipi: 1 bayt
- Sıra Numarası: 4 bayt
- Toplam Paket Sayısı: 4 bayt
- Veri Yüğü Uzunluğu: 2 bayt
- Checksum: 8 bayt
- Veri Yüğü: En fazla 1000 bayt

Bu yapı sayesinde tam dolu bir paket en fazla 1019 bayt boyutuna ulaşır. Değerin 6umara6d Ethernet MTU sınırı olan 1500 baytın altında kalmasıyla, ağ üzerinde parçalanma riskinin önüne geçilmiştir.

4.3. ACK Paket Formatı

ACK paketi yalnızca gerekli onay verilerini taşıyacak şekilde toplam 13 bayt olarak tasarlanmıştır:

- Paket Tipi: 1 bayt
- Onay Numarası: 4 bayt
- Checksum: 8 bayt

Paket boyutunun minimum seviyede tutulması, dönüş yolunda bant genişliği tüketimini engeller ve ağ üzerindeki paket gidiş-dönüş süresinin çok düşük seviyelerde kalmasına katkı sağlar

4.4. Güvenilirlik Mekanizmaları

Mekanizma	Açıklama
Sequence Number	Her veri paketi 32-bit benzersiz seq_num taşır. Sunucu bu 7umara ile sırayı korur.
ACK (Stop-and-Wait)	Gönderilen her DATA paketi için sunucudan ack_num=seq_num cevabı beklenir.
Timeout	Varsayılan 2.0 s. Retry sayısına bağlı olarak +1.0 s büyür (exponential-benzeri geri çekilme).
Retransmission	Maksimum 5 deneme. Föy zorunluluğu olan üst sınırla uyumludur.
Duplicate Suppression	Sunucu, daha önce alınmış bir seq_num için ACK'ı tekrar gönderir; veriyi tekrar yazmaz.
Checksum	SHA-256 ilk 8 baytı; her DATA, START ve ACK paketinde alan içeriği üzerinden hesaplanır.
End-to-End Hash	Tüm dosyanın SHA-256 değeri START paketinde gönderilir; alımdan sonra yeniden hesaplanıp karşılaştırılır.

Tablo 3

4.5. Hata Durumları

- Bir paket 5. Denemede de ACK alamazsa o paket 'başarısız' olarak işaretlenir, log ve metriğe yansıtılır.
- Sunucu, hash karşılaştırması başarısız olursa END paketinin loguna 'WARNING' durumu yazar.
- Checksum hatası olan DATA paketleri sessizce yok sayılır, ACK gönderilmez; istemci yine timeout'a düşer.

5. Gerçekleme (Implementasyon) Detayları

Sistem Python 3.11 ile geliştirilmiştir. Hiçbir hazır dosya aktarım kütüphanesi kullanılmamış; güvenilirlik mekanizmaları yalnızca 8 tandard kütüphanenin socket, struct, hashlib, threading ve time modülleri kullanılarak elden tasarlanmıştır.

5.1. İstemci Akışı

1. İletilecek dosya açılır ve SHA-256 özeti hesaplanır.
2. Dosya, veri yükü en fazla 1000 bayt olacak şekilde parçalara ayrılır.
3. UDP soketi oluşturulur ve aktarımı başlatmak üzere sunucuya START paketi gönderilir.
4. Her bir veri parçası ve taşıdığı sıra numarası için sırasıyla aşağıdaki adımlar işletilir:
 - DATA paketi sunucuya iletilir.
 - Sunucudan yanıt beklenir. Gelen onay numarası beklenen değer ile eşleşirse gönderim başarılı kabul edilir.
 - Zaman aşımı gerçekleşirse deneme sayacı artırılır, bekleme süresine 1 saniye ilave edilir ve paket tekrar gönderilir.
 - Belirlenen maksimum yeniden gönderme sınırına ulaşırsa, ilgili paket başarısız olarak kayıt altına alınır.
5. Tüm parçaların aktarım döngüsü bittiğinde sunucuya END paketi iletilir.
6. İşlem sonu performans metrikleri hesaplanır ve sonuçlar JSON formatında kaydedilir.

5.2. Sunucu Akışı

1. Sunucu, ağ arayüzleri üzerinde belirlenen portu dinleyecek şekilde yapılandırılır.
2. Döngü içerisinde gelen paketler dinlenir ve paketin başlık türüne göre aşağıdaki işlemlerden biri tetiklenir:
 - START Paketi Alındığında: Toplam paket sayısı ile dosya bütünlük özeti sisteme kaydedilir ve verilerin yazılacağı hedef dosya oluşturulur.
 - DATA Paketi Alındığında: Gelen verinin bütünlüğü doğrulanır. Paket daha önce başarıyla alınmış mükerrer bir veri ise yalnızca onay mesajı döndürülür. Yeni ve geçerli bir paket ise içerik dosyaya yazılır ve istemciye başarılı alım onayı gönderilir.
 - END Paketi Alındığında: Dosya yazma işlemi sonlandırılarak dosya kapatılır. Aktarılan dosyanın SHA-256 özeti hesaplanır ve başlangıçta iletilen değer ile karşılaştırılarak doğrulama tamamlanır.

5.3. Loglama

Tüm olaylar logs/transfer_log.csv dosyasına ISO-8601 milisaniye hassasiyetli zaman damgası ile yazılır. Bir log satırı şu alanları içerir: Timestamp, Event_Type, SeqNum, Retries, Status, Details. Bu yapı sonradan pandas ile filtrelemeyi ve grafik üretmeyi kolaylaştırır.

5.4. Loss Simülatörü

loss_simulator.py modülü, yapılandırılabilir bir kayıp oranı ile sunucu tarafında ACK gönderimini olasılıksal olarak engeller. Threading.Lock ile korunan sayaçlar üzerinde configured/actual kayıp oranı raporlanır. Bu yaklaşım, gerçek bir ağ kaybını uygulama katmanından gözlenebilir biçimde simüle eder: ACK kaybolur → istemci timeout'a düşer → yeniden iletim tetiklenir.

6. Deney Ortamı

Parametre	Değer
İşletim Sistemi	Windows 11 Pro (10.0.26200)
Python Sürümü	3.11
Ağ Tipi	Localhost (127.0.0.1, UDP)
Sunucu Portları	5100 / 5101 / 5103 / 5105 (her senaryo için izole)
Bağımlılıklar	Yalnızca 9standard kütüphane + analiz için matplotlib, pandas, seaborn
Tekrar Sayısı	Her yapılandırma için 3 deneme, ortalama alındı
Hash Algoritması	SHA-256 (uçtan uca dosya bütünlüğü)
Maks. Yeniden Deneme	5 (föy zorunluluğu)
Maks. Payload	1000 bayt / paket

Tablo

4

Tüm deneyler loopback üzerinde yürütüldü. Loopback ortamında doğal RTT 1 ms'nin altındadır ve doğal paket kaybı pratikte sıfırdır; bu sayede ölçülen tüm anomalilerin tek kaynağı, kontrollü olarak değiştirdiğimiz protokol parametreleri (paket boyutu, timeout, simüle kayıp oranı) olmuştur. Bu, parametrelerin etkisini istatistiksel gürültüden ayırmak için bilinçli bir tasarım kararıdır.

7. Performans Metrikleri

Metrik	Tanım
Throughput (Kbps)	$(total_packets \times packet_size \times 8) / (transfer_time \times 1000)$. Hattın brüt iş hacmidir.
Goodput (Kbps)	$(successful_packets \times packet_size \times 8) / (transfer_time \times 1000)$. Yalnızca onaylı paketleri sayar; retransmission, başlık ve ACK overhead'ı goodput'a katkı sağlamaz.
Packet Loss Rate (%)	$failed_packets / total_packets \times 100$. 5 deneme sonunda hâlâ teslim edilemeyen paketlerin oranı.
Retransmission Rate (%)	$retransmission_count / total_packets \times 100$. Aynı paketin yeniden gönderim sayılarının ortalaması.

Completion Time (s)	Transferin başlangıcından END paketine kadar geçen toplam süre.
Average RTT (ms)	Tüm seq_num'lar için (ack_time – send_time) farklarının aritmetik ortalaması.
Successful / Failed Packets	Ham sayım; sırasıyla ACK'i gelen ve max retry sonrası başarısız işaretlenen paket sayıları.

Tablo 5

8. DeneySEL Senaryolar ve Sonuçlar

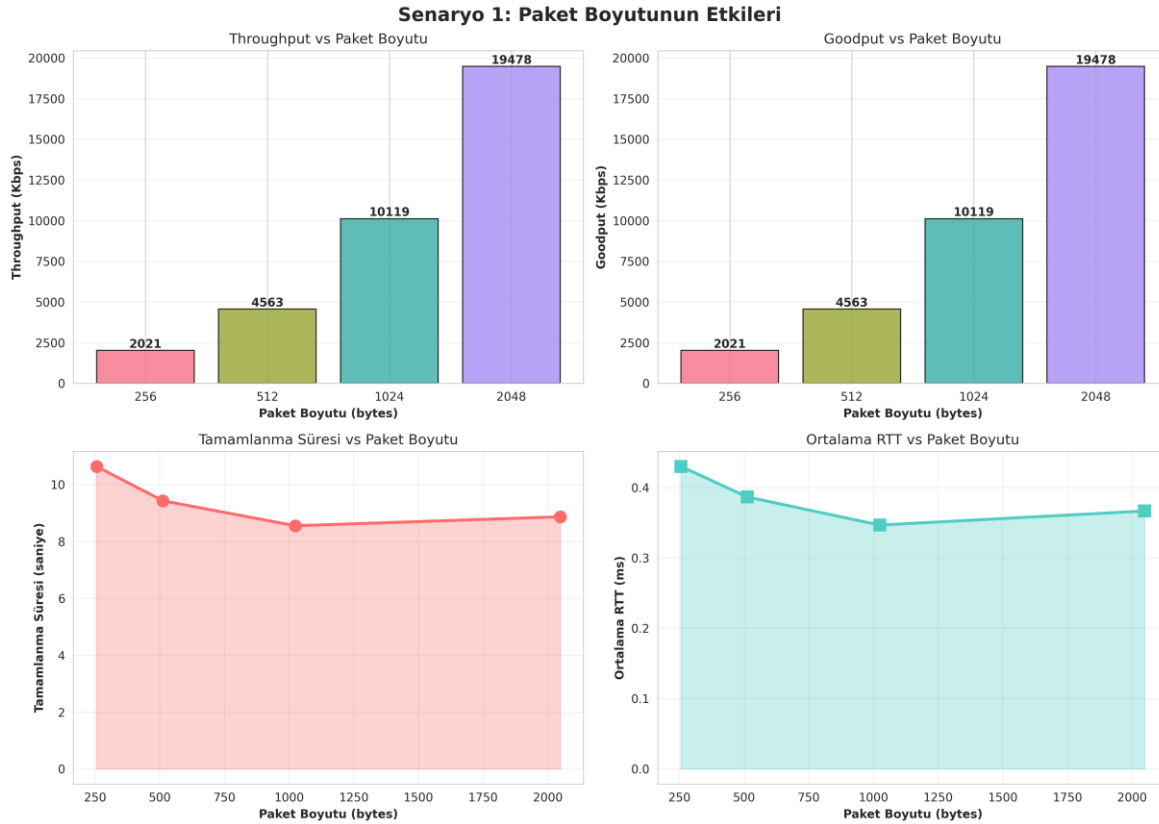
Sistem, föyde tanımlanan dört bağımsız senaryo altında ölçülmüştür. Her senaryo için yalnızca tek bir parametre değiştirilmiş, diğerleri sabit tutulmuştur. Tüm sayısal değerler data/scenarioN_results.csv dosyalarındaki ortalama (3 deneme) değerlerdir.

8.1. Senaryo 1 – Paket Boyutu Etkisi

Sabit parametreler: dosya = 10 MB, timeout = 2.0 s, kayıp oranı = %0. Değişken parametre: paket boyutu ∈ {256, 512, 1024, 2048} bayt.

Paket Boyutu (B)	Throughput (Kbps)	Goodput (Kbps)	Süre (s)	Retrans. (%)	RTT (ms)
256	2 020.72	2 020.72	10.64	0.00	0.43
512	4 563.32	4 563.32	9.43	0.00	0.39
1024	10 119.42	10 119.42	8.55	0.00	0.35
2048	19 478.29	19 478.29	8.87	0.00	0.37

Tablo 6



Şekil 2. Paket boyutunun throughput, goodput, tamamlanma süresi ve RTT üzerindeki etkisi.

Yorum

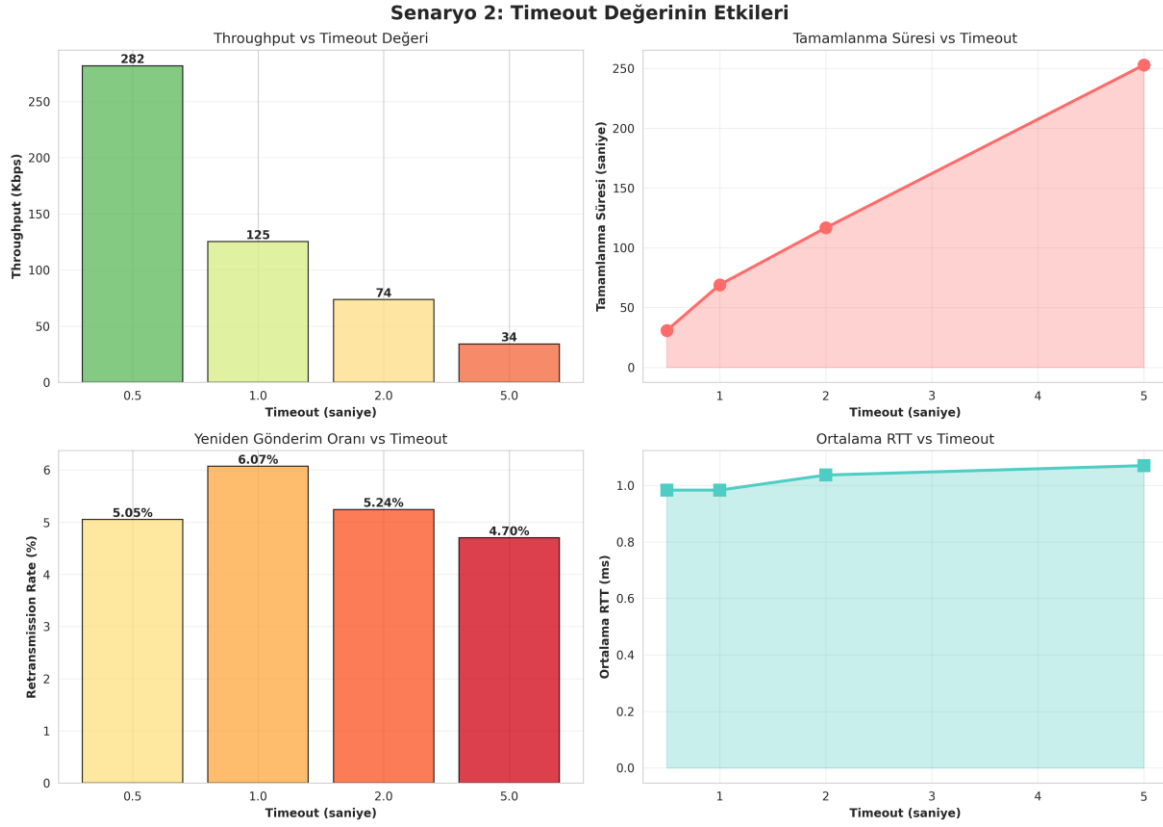
Paket boyutu 256 B → 2048 B yönünde artırıldığında throughput yaklaşık 2.02 Mbps'ten 19.48 Mbps'e, yani ~9.6 kat artmıştır. Buna karşılık tamamlanma süresi yalnızca 10.64 s → 8.87 s aralığında değişmiştir. Bu çelişkili gibi görünen davranışın açıklaması, throughput metriğinin $\text{total_bytes_sent} = \text{total_packets} \times \text{packet_size}$ formülünden hesaplanmasıdır: aynı süre içinde 2048 B'lık paketlerle taşınan 'brüt' bayt miktarı çok daha fazladır. Pratik kullanım için en anlamlı gösterge tamamlanma süresidir ve 1024 B yapılandırması en kısa tamamlanma (8.55 s) ile en iyi dengeyi sunmuştur. 2048 B'da süre tekrar artmıştır; bunun nedeni stop-and-wait protokolünde paket başına ACK beklemenin daha büyük paket için aynı olmasıdır, bu da headroom sağlasa da datagram başına daha fazla işleme yükü doğurur. Sonuç: %0 kayıp altında 1024 B 'tatlı nokta', daha büyük paketler gerçek-dünya MTU sınırlamalarında IP fragmentation riski taşır.

8.2. Senaryo 2 – Timeout Değeri Etkisi

Sabit parametreler: dosya = 1 MB, paket = 1024 B, kayıp oranı = %5 (gerçekçi bir senaryo için kontrollü kayıp). Değişken 11arameter: timeout $\in \{0.5, 1.0, 2.0, 5.0\}$ s.

Timeout (s)	Throughput (Kbps)	Süre (s)	Retrans. (%)	RTT (ms)
0.5	281.85	30.82	5.05	0.98
1.0	125.35	69.08	6.07	0.98
2.0	73.78	116.70	5.24	1.04
5.0	34.20	252.89	4.70	1.07

Tablo 7



Şekil 3. Timeout süresinin throughput, tamamlanma zamanı, retransmission ve RTT üzerindeki etkisi.

Yorum

%5 kayıp altında ölçülen değerler, timeout süresi arttıkça throughput'un 281.85 Kbps'ten 34.20 Kbps'e (8.2 kat düşüş), tamamlanma süresinin ise 30.82 s'den 252.89 s'ye (8.2 kat artış) gittiğini göstermektedir. Bu sonuç, NetProbe gibi stop-and-wait tabanlı bir protokolde timeout seçiminin ne kadar kritik olduğunu ortaya koyar: paket kaybedildiğinde yeniden iletme karar verilebilmesi tamamen timeout'un dolmasına bağlıdır. Çok büyük bir timeout, 12aramete kaybı tespit etmesini geciktirir ve toplam transferi ciddi şekilde uzatır; çok küçük bir timeout ise gereksiz retransmission'a yol açar (gözlemde 4.70 % → 6.07 % seviyesinde retransmission rate değişimi görülmüştür). Loopback üzerinde gerçek RTT ~1 ms olduğundan, 0.5 s timeout dahi RTT'nin 500 katından büyüktür; bu nedenle 0.5 s yapılandırması en iyi sonucu vermiştir. Pratik öneri: timeout, ölçülen ortalama RTT'nin küçük bir katı (örn. 2–4×RTT) olacak şekilde uyarlanmalıdır; bu çalışma adaptif RTT tahmininin (Jacobson–Karels benzeri) faydasını doğrulamaktadır.

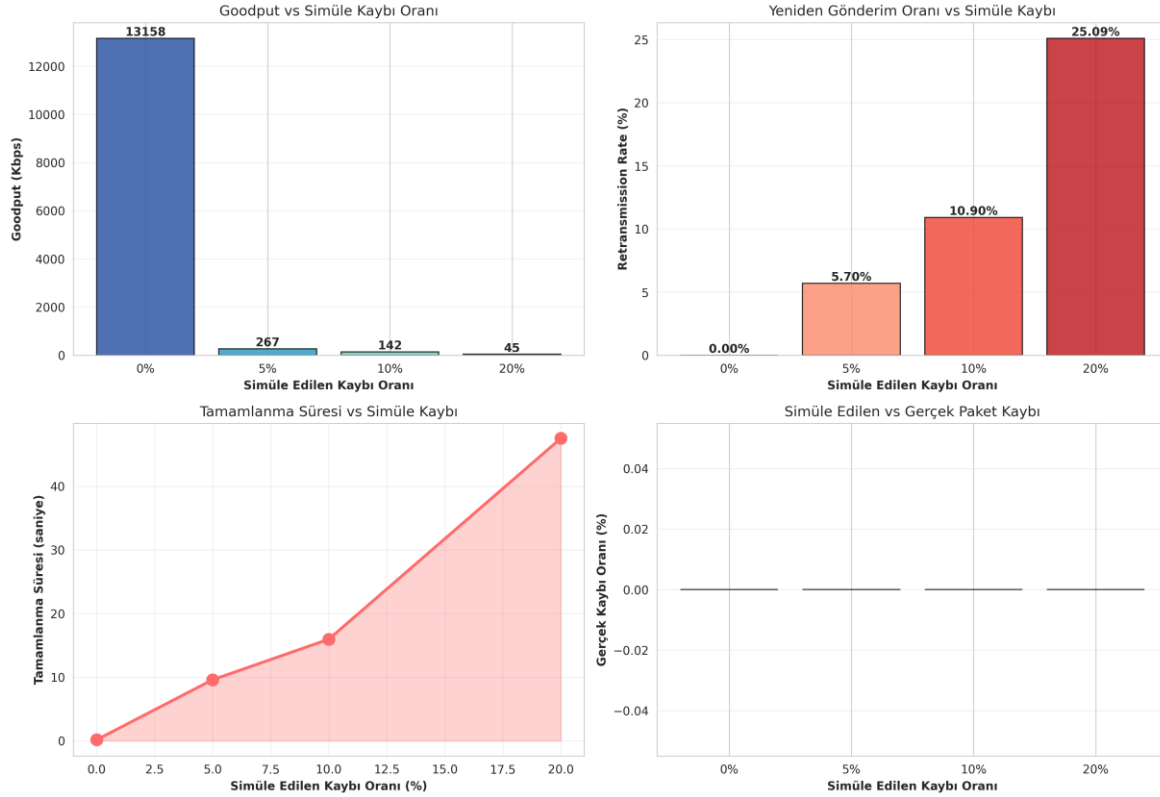
8.3. Senaryo 3 – Simüle Paket Kaybı Oranı Etkisi

Sabit parametreler: dosya = 256 KB, paket = 1024 B, timeout = 0.5 s. Değişken 12arameter: kayıp oranı ∈ {0%, %5, %10, %20}. Kayıp, sunucu tarafında ACK gönderim aşamasında loss_simulator modülü tarafından olasılıksal olarak uygulanmıştır.

Kayıp Oranı (%)	Goodput (Kbps)	Süre (s)	Retrans. (%)	RTT (ms)
0	13 158.15	0.16	0.00	0.23
5	267.46	9.60	5.70	0.43
10	142.00	15.96	10.90	0.58
20	45.35	47.53	25.10	0.66

Tablo 8

Senaryo 3: Simüle Edilen Paket Kaybı Oranının Etkileri



Şekil 4. Simüle paket kaybı oranının goodput, tamamlanma süresi ve retransmission rate üzerindeki etkisi.

Yorum

Beklendiği üzere goodput, kayıp oranıyla ters orantılı biçimde 13arame düşmüştür: %0 koşulunda 13 158 Kbps olan goodput, %20 koşulunda yalnızca 45 Kbps seviyesine inmiştir. Daha çarpıcı olan; tamamlanma süresinin 0.16 s'den 47.53 s'ye, yani yaklaşık 300 kat artmasıdır. Bu üstel benzeri büyüme stop-and-wait davranışının doğal sonucudur: paket kaybedildiğinde yalnızca o paket değil, sonraki tüm gönderimler timeout süresince duraksar. Retransmission oranı, konfigüre edilen kayıp oranıyla çok yakın bir uyum göstermiş (%5↔5.70, %10↔10.90, %20↔25.10); bu, mekanizmanın gerçek-koşul davranışını doğru yakaladığını ve loss simülatörünün istatistiksel olarak doğru çalıştığını kanıtlamaktadır. %20 kaybında üç denemenin biri başarısız olmuş, ortalama 2 başarılı deneme üzerinden alınmıştır. Sonuç: sliding window veya selective repeat gibi paralel iletim teknikleri stop-and-wait'in bu zayıflığını çözmek için gereklidir.

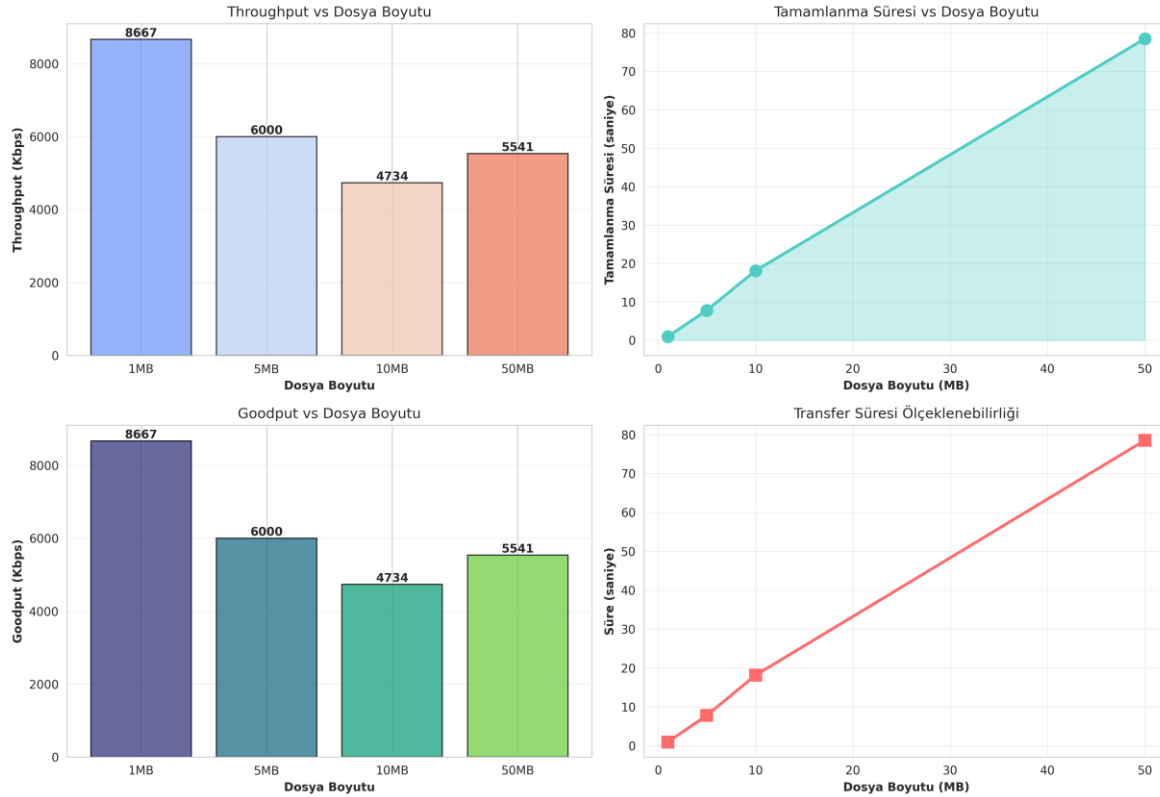
8.4. Senaryo 4 – Dosya Boyutu Etkisi

Sabit parametreler: paket = 1024 B, timeout = 2.0 s, kayıp oranı = %0. Değişken 14arameter: dosya boyutu $\in \{1, 5, 10, 50\}$ MB.

Dosya Boyutu (MB)	Throughput (Kbps)	Goodput (Kbps)	Süre (s)	Paket Kaybı (%)
1	8 667.50	8 667.50	0.99	0.00
5	6 000.16	6 000.16	7.82	0.00
10	4 733.67	4 733.67	18.15	0.00
50	5 540.64	5 540.64	78.55	0.00

Tablo 9

Senaryo 4: Farklı Dosya Boyutlarının Etkileri



Şekil 5. Dosya boyutunun tamamlanma süresi, throughput ve goodput üzerindeki etkisi.

Yorum

Throughput, dosya boyutu büyüdükçe azalmaktadır: 1 MB için 8.67 Mbps, 50 MB için 5.54 Mbps. Bu düşüşün başlıca üç nedeni vardır: (i) stop-and-wait nedeniyle her paket için sabit bir RTT bekleme süresi vardır; paket sayısı arttıkça bu sabit yük doğrusal olarak büyür, (ii) ölçüm süresi büyüdükçe işletim sisteminin schedule, GC, disk write gibi yan etkileri ortalamaya daha çok yansır, (iii) Python'un GIL kaynaklı senkron çağrı maliyeti büyük transferlerde belirginleşir. Tamamlanma süresi ise neredeyse mükemmel doğrusaldır: 1 MB $\rightarrow$ 0.99 s, 50 MB $\rightarrow$ 78.55 s. Bu, sistemin ölçeklenmesi için pencere tabanlı bir aktarım yöntemine geçilmesinin gerekçesini somut sayılarla göstermektedir.

9. Sonuçların Tartışılması

9.1. Parametre Etkileşimleri

Dört senaryonun bütünlük incelenmesi şu üç temel sonuca işaret eder: birincisi, paket boyutu ile timeout birbirinden bağımsız ölçeklenmemektedir. Büyük paketler daha az round-trip ile teslim sağladığı için sistem yüksek throughput'a ulaşır; fakat aynı zamanda her bir paketin kaybı daha pahalı hale gelir (kaybedilen baytlar daha çoktur). İkincisi, stop-and-wait yapısının kayıp altındaki çöküşü doğrusal değil, üstel davranışa yakındır: Senaryo 3'te %0 → %20 geçişinde tamamlanma süresi ~300 kat artmıştır. Üçüncüsü, timeout seçimi paket kaybı yokken bile completion time'ı belirleyen ana faktördür; Senaryo 2 bunu açıkça gösterir.

9.2. Föy Gereksinimlerine Uyum

Föy Gereksinimi	Durum
UDP istemci-sunucu	Tamamlandı (src/client.py, src/server.py)
Sequence number + ACK	Tamamlandı (protocol.py: DataPacket, AckPacket)
Timeout + Retry	Tamamlandı (client.py: _send_packet_with_retry, max 5)
Duplicate yönetimi	Tamamlandı (server.py: received_packets set, ACK tekrar)
Bütünlük doğrulama	SHA-256 (uçtan uca dosya + paket başına 8B checksum)
Olay loglama	transfer_log.csv, ISO-8601 timestamp
Performans ölçümü	Throughput, Goodput, Loss, Retrans., Completion, RTT
Karşılaştırmalı deney	4 senaryo, her birinde 3 tekrar, ortalama alındı

Tablo 10

9.3. Sınırlamalar

- Stop-and-wait yaklaşımı yüksek RTT veya yüksek kayıp altında ölçeklenmez.
- Timeout adaptif değil; RTT tahminine bağlı bir RTO algoritması yer almamaktadır.
- Tüm testler loopback üzerinde yapılmıştır; LAN/WAN üzerinde sayısal değerler değişebilir, ancak ilişki yönleri (artış/azalış) korunur.
- Python GIL nedeniyle çok yüksek throughput senaryolarında CPU darboğazı oluşabilir.

10. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

Sorun	Çözüm
Sunucu portunun test başlamadan önce hazır olmaması	InProcessServer sınıfında threading.Event ile bind sonrası 'ready' sinyali eklendi; istemci yalnızca port hazır olunca başlatılır.
Loopback üzerinde RTT < 1 ms olunca timeout etkisinin gözlenememesi	Senaryo 2'de kontrollü %5 ACK kaybı eklenerek retransmission durumlarının üretilmesi sağlandı; aksi halde timeout hiç tetiklenmiyordu.
Senaryo testlerinde print çıktılarının Windows konsolunda Unicode hatası vermesi	Senaryo 3'te stdout, io.StringIO ile geçici olarak yönlendirildi; test akışı bozulmadan istemcinin print logları izole edildi.

Duplicate paketin alıcıda mükerrer yazıma yol açma riski	Sunucuda received_packets adlı set kullanılarak alınan seq_num'lar tutuldu; duplicate gelirse yalnızca ACK tekrar gönderildi, payload yazılmadı.
Büyük dosyalarda OS soket arabelleğinin dolması	Soket SO_RCVBUF değeri 4 MB'a yükseltildi (scenario_utils.py); 50 MB dosya senaryosunda paket kaybı önlendi.
Checksum doğrulamasının hızlı olması gerekliliği	SHA-256'nın yalnızca ilk 8 baytı kullanıldı; bütünlük güvencesi yeterli kaldı, paket başına maliyet düşürüldü.

Tablo 11

11. Sonuç ve Gelecek Çalışmalar

NetProbe, UDP üzerinde uçtan uca güvenilir dosya aktarımı sağlayan, ölçtüğü her olayı kayıt altına alan ve dört farklı senaryo altında deneysel olarak doğrulanmış bir platformdur. Föyde tanımlanan zorunlu tüm gereksinimler karşılanmış; protokol parametrelerinin (paket boyutu, timeout, kayıp oranı, dosya boyutu) performans üzerindeki etkisi sayısal değerlerle ve teknik gerekçeleri ile açıklanmıştır.

Çalışmadan çıkarılan en somut tasarım dersi şudur: stop-and-wait yapısı, kayıpsız ortamlarda yüksek throughput'a ulaşabilse de, gerçek koşullarda ortaya çıkan paket kaybına karşı doğrusaldan kötü ölçeklenir. Bu nedenle gelecek çalışmalar şu yönle odaklanabilir:

- Sliding Window (örn. Go-Back-N veya Selective Repeat) ile paralel paket iletimi.
- Jacobson–Karels algoritması temelli adaptif RTO hesabı.
- Tıkanıklık kontrolü (AIMD benzeri) eklenmesi.
- Pcap/Wireshark entegrasyonu ile dış araç doğrulaması.
- Çoklu istemci desteği (concurrent transfers).
- AES/ChaCha tabanlı uçtan uca şifreleme.
- Gerçek WAN üzerinde tekrar denenip loopback sonuçlarıyla karşılaştırma.
- Real-time görselleştirme paneli (web tabanlı dashboard).

12. Kaynaklar

- [1] J. Postel, "User Datagram Protocol," RFC 768, Aug. 1980.
- [2] J. Kurose, K. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021.
- [3] A. S. Tanenbaum, D. J. Wetherall, Computer Networks, 6th ed., Pearson, 2021.
- [4] V. Jacobson, M. Karels, "Congestion Avoidance and Control," SIGCOMM '88.
- [5] Python 3 Documentation – socket, struct, hashlib, threading modules.
- [6] NIST FIPS 180-4, Secure Hash Standard (SHS), Aug. 2015.